



Customer Support Ticket Classifier

AI-Powered Ticket Triage System

Task 29 — Final Capstone Project

PKCERT AI & Software Development Internship

Submitted by: Abdul Rehman

GitHub: github.com/Abdul1652005/TASK-29

August 2026

1. Project Overview

This project is a machine learning system that automatically reads a customer support ticket and predicts which category it belongs to — such as Hardware, Access, Billing-related issues, or HR Support. In a real company, support tickets are usually sorted manually by a person, which takes time and delays response. This tool automates that first step, so tickets can be routed to the right team faster.

Live Links

Resource	Link
Live Frontend (Streamlit)	https://project-findit-qdnpmcct2jftkub8sy2ygt.streamlit.app/
Live Backend API (Railway)	https://task-29-production.up.railway.app
API Docs (Swagger)	https://task-29-production.up.railway.app/docs
GitHub Repository	https://github.com/Abdul1652005/TASK-29

2. Problem Statement

Target User

Support teams at a company that receive many customer tickets every day and need to sort them into categories before assigning them to the right department.

Core Functionality

- User types or pastes a ticket description into a text box.
- The system predicts the most likely category for that ticket.
- The system shows how confident it is in that prediction.
- A history of past predictions is kept for the session.

Value Provided

Reduces the time spent manually reading and sorting tickets, and reduces human error in routing tickets to the wrong department.

3. Project Scope

In Scope

- Text classification of a single ticket at a time
- 8 predefined categories (Access, Administrative rights, HR Support, Hardware, Internal Project, Miscellaneous, Purchase, Storage)
- A web-based interface (Streamlit) with a Classify tab, History tab, and About tab
- A REST API (FastAPI) that the frontend calls for predictions
- Session-based history tracking and CSV export

Out of Scope

- User accounts / login system (not needed for this use case)
- Automatic ticket routing to real support software (e.g. Zendesk integration)
- Multi-language support (English only)
- Real-time monitoring dashboards or analytics beyond session history

4. Technology Stack & Justification

Layer	Technology	Reason
Model Training	Jupyter Notebook	Simple, step-by-step, easy to document and rerun
Model	TF-IDF + Linear SVM	Lightweight, fast, works well on limited hardware (no GPU needed)
Backend API	FastAPI	Fast to build, automatic docs, easy input validation
Frontend	Streamlit	Quick to build a clean UI without separate HTML/CSS/JS work
Backend Hosting	Railway	Free tier, simple GitHub-based deployment
Frontend Hosting	Streamlit Community Cloud	Free, no credit card required, direct GitHub deploy

A Transformer-based model (e.g. BERT) was considered, but a classical ML model (TF-IDF + Linear SVM) was chosen instead because it trains and runs quickly on a standard laptop with no GPU, and it still gives strong, reliable accuracy for this type of text classification task.

5. Dataset & Data Preparation

Data Source

A public, labeled customer support ticket dataset (all_tickets_processed_improved_v3.csv), containing ticket descriptions and their true category labels. Licensing allows free use for educational and research purposes.

Cleaning Steps

- Removed rows with missing ticket text or missing category label
- Converted all text to lowercase and trimmed extra spaces
- Removed empty ticket entries after cleaning

Dataset Size

47,837 tickets total across 8 categories, split into training (70%), validation (15%), and test (15%) sets using stratified sampling, so each category is fairly represented in every split.

6. Model Building & Evaluation

Approach

Ticket text was converted into numerical features using TF-IDF (Term Frequency–Inverse Document Frequency), which measures how important a word is to a specific ticket compared to all other tickets. Two models were then trained and compared on the same features.

Baseline Model: Logistic Regression

Metric	Score
Accuracy	84.9%
Precision	85.5%
Recall	84.9%
F1 Score	84.9%

Improved Model: Linear SVM (LinearSVC)

Metric	Score
Accuracy	85.5%
Precision	85.7%
Recall	85.5%
F1 Score	85.5%

The Linear SVM gave a small but consistent improvement over the Logistic Regression baseline across every metric, so it was selected as the final model for deployment.

Error Analysis

Most misclassifications happened between categories with overlapping vocabulary — for example, tickets mentioning both a device and an account issue could be confused between 'Hardware' and 'Access'.

Categories with fewer training examples (like 'Administrative rights') also had a slightly higher error rate, which is expected with less data per class.

7. System Architecture

The system follows a simple three-layer flow:

- Jupyter Notebook — data cleaning, model training, evaluation, and saving the final model (ticket_model.joblib) and vectorizer (ticket_vectorizer.joblib)
- FastAPI Backend — loads the saved model, exposes a POST /predict endpoint that accepts ticket text and returns the predicted category, confidence score, and full score breakdown
- Streamlit Frontend — provides the user interface, sends the entered ticket text to the backend API, and displays the result

Backend API Contract

Field	Type	Description
Request: text	string	The ticket description entered by the user
Response: category	string	Predicted category label
Response: confidence	float	Relative confidence percentage (0–100)

Field	Type	Description
Response: distribution	object	Confidence score for every category

Input validation ensures empty or missing text is rejected with a clear error message rather than being sent to the model.

8. Application Screenshots

8.1 Classify Tab

The main interface where a user enters a ticket description and receives a predicted category with a confidence score and a breakdown across all categories.

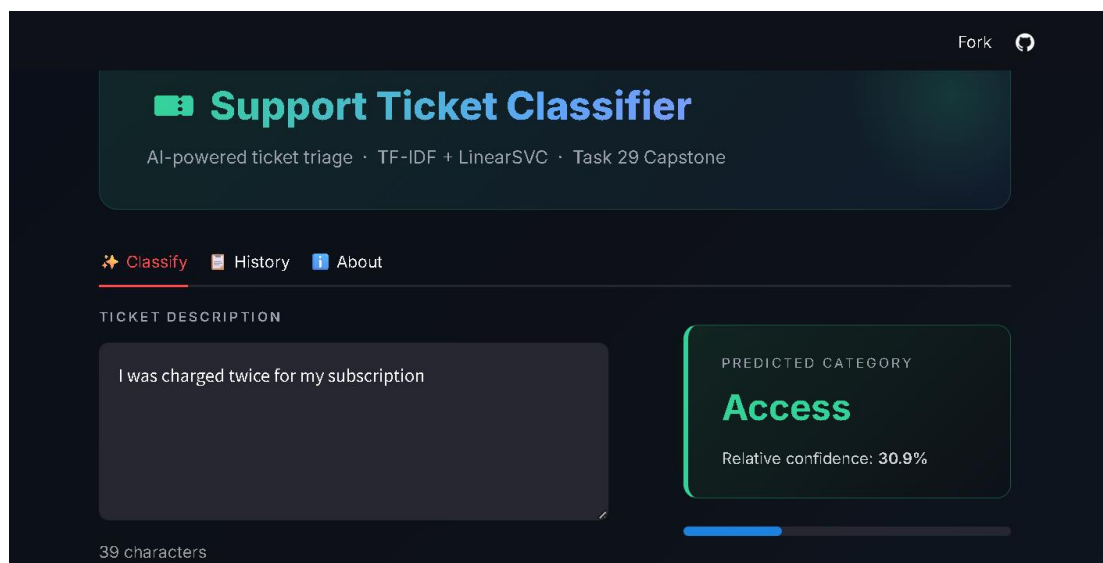


Figure 1 — Classify tab showing a prediction for a billing-related ticket.

8.2 History Tab

Tracks every classification made during the current session, along with a distribution chart of categories seen so far. Includes a CSV export option.

Ticket Snippet	Category	Confidence	Time
My laptop screen is black and won't turn on	Hardware	67.2%	06:46:07
I can't reset my password	Access	95.7%	06:45:58
The app crashes every time I open it	HR Support	53.8%	06:45:48
I was charged twice for my subscription	Access	30.9%	06:45:02
I need to buy new Washing Machine.	Hardware	33.9%	06:43:22

Figure 2 — Session history of classified tickets.

8.3 About Tab

Describes the project, tech stack, and supported categories for transparency.

Support Ticket Classifier
AI-powered ticket triage · TF-IDF + LinearSVC · Task 29 Capstone

✦ Classify ▢ History ▢ About

PROJECT

Customer Support Ticket Classifier built as the **Task 29 Final Capstone** of the PKCERT AI & Software Development Internship.

Automatically categorizes free-text support tickets using a trained machine learning model, reducing manual triage time and improving response routing.

TECH STACK

Model	TF-IDF + LinearSVC
Backend	FastAPI + Uvicorn
Frontend	Streamlit
Training	Jupyter Notebook
Serialization	joblib

SUPPORTED CATEGORIES

Access Administrative rights HR Support Hardware Internal Project Miscellaneous Purchase Storage

HOW IT WORKS

- Text is lowercased & cleaned
- TF-IDF converts text → feature vector
- LinearSVC produces decision scores
- Softmax applied for relative confidence
- Top category returned as prediction

AUTHOR

Abdul Rehman
github.com/Abdul1652005

Figure 3 — About tab with project and tech stack details.

8.4 Sidebar — Model Info

Shows live model architecture, supported categories, and the data pipeline at a glance.

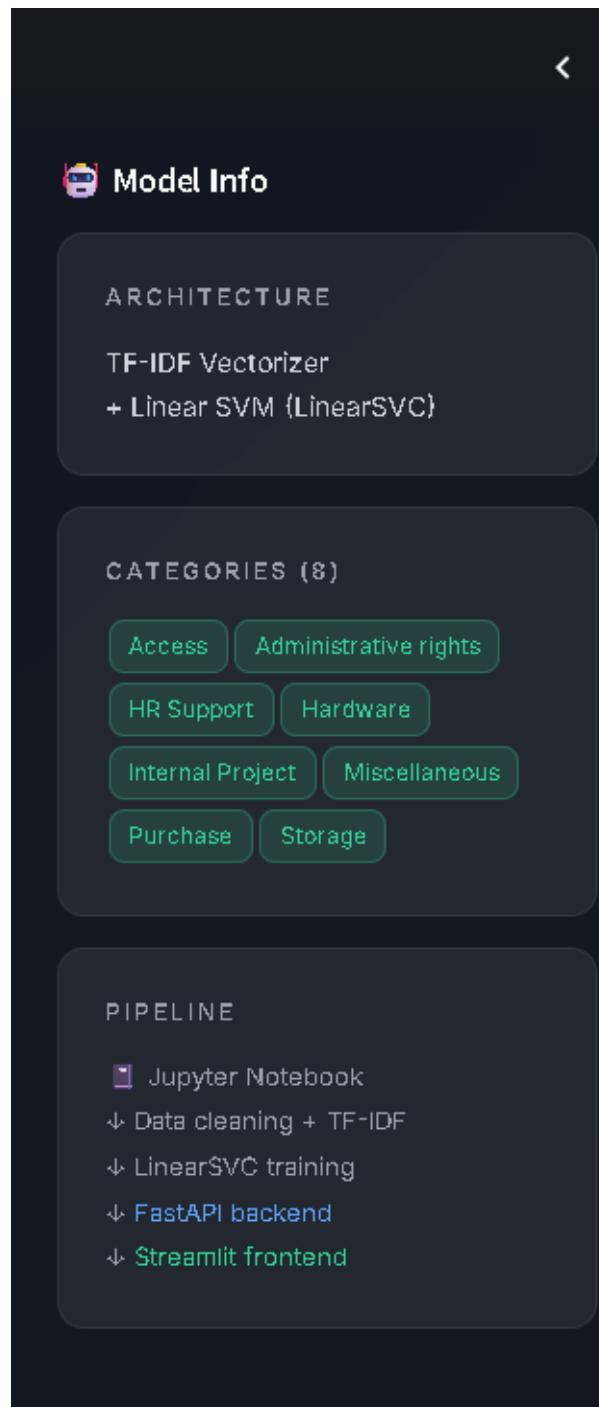


Figure 4 — Sidebar showing model architecture and pipeline.

8.5 FastAPI Interactive Docs

FastAPI automatically generates interactive API documentation, allowing the `/predict` endpoint to be tested directly in the browser.

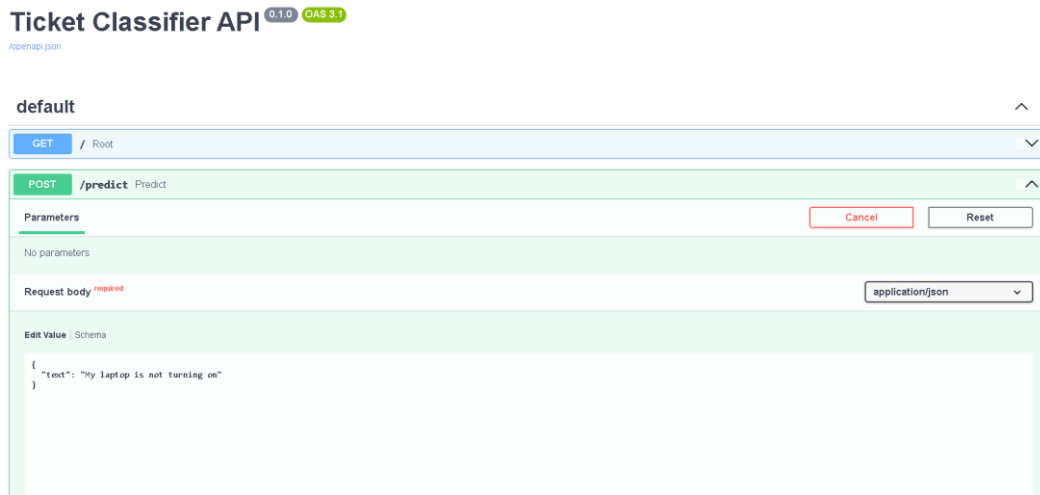


Figure 5 — FastAPI Swagger UI showing the /predict endpoint.

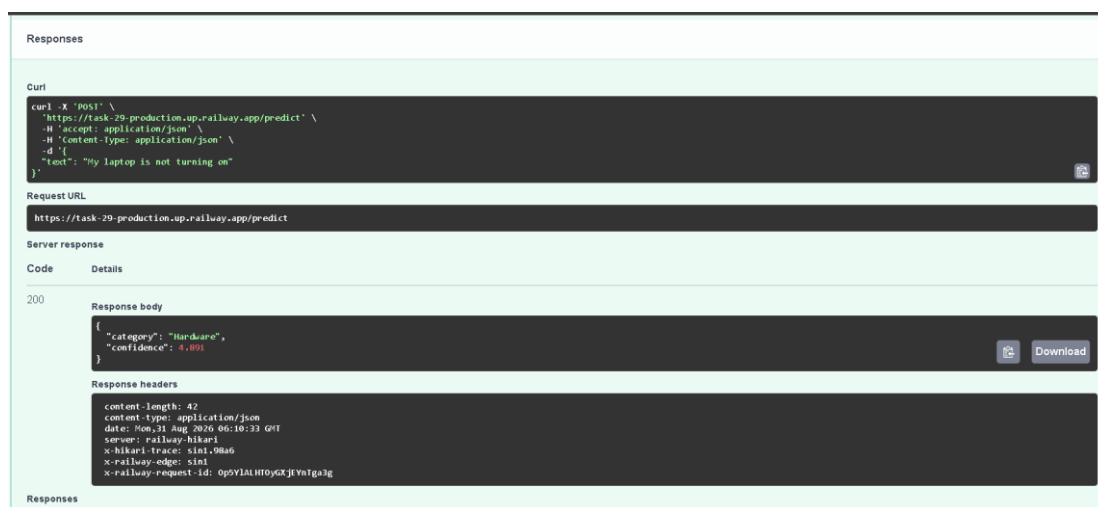


Figure 6 — Successful 200 OK response from the live Railway-hosted API.

9. Testing

Test Case	Input	Expected Result	Actual Result
Valid ticket text	"My laptop is not turning on"	Returns a predicted category	Passed — returned 'Hardware'
Empty text	""	Rejected with validation error	Passed — 422 error returned
Missing field	{ } (no text field)	Rejected with validation error	Passed — 422 error returned
API health check	GET /	Returns status ok	Passed — {"status": "ok"}
Live API call from frontend	Any valid ticket	Frontend displays result from backend	Passed — verified via screenshots

10. Deployment

- Backend: Deployed on Railway, connected directly to the GitHub repository for automatic redeployment on every push.
- Frontend: Deployed on Streamlit Community Cloud, also connected to GitHub for automatic redeployment.

- Both services are free-tier and required no credit card.

Deployment Notes

An early build issue occurred when Streamlit Cloud defaulted to Python 3.14, which lacked prebuilt packages for some dependencies. This was resolved by explicitly setting the Python version to 3.11 in the deployment settings.

11. Challenges Faced

- Hardware limits: training had to stay lightweight (TF-IDF + classical ML) rather than a heavier Transformer model, to run comfortably on a standard laptop.
- Free hosting constraints: Railway's free tier required avoiding memory-heavy models; Streamlit Cloud needed a Python version fix to build successfully.
- Dataset selection: an initial dataset produced near-random results and had to be replaced with a better-labeled dataset.
- Connecting frontend and backend reliably, including handling cases where the API might be temporarily unreachable.

12. Conclusion & Lessons Learned

This project delivered a complete, working, end-to-end machine learning application — from raw data to a live, publicly accessible tool. Key takeaways include the importance of choosing the right model for the available hardware, the value of testing each layer (model, API, UI) independently before connecting them, and how free-tier cloud platforms can still support a fully functional deployment when configured correctly.